

Product Requirements Document

Dynamic Forecast of Expected Time of Arrival (ETA) for Coaching Trains

Problem Statement ID	26028	Category	Software
Organization	Ministry of Railways	Theme	Smart Automation
Build Window	3 days (hackathon MVP)	Doc Version	v1.0

1. Problem Summary & Goal

Indian Railways currently estimates ETA using static schedules plus current delay and a fixed recovery-time buffer. This ignores real-time ground conditions — congestion, unscheduled halts, speed restrictions, weather, and cascading effects from preceding trains — so published ETAs drift from reality, hurting passengers, station operations, crew/platform planning, and feeder logistics.

Goal for this build: demonstrate a working, data-driven ETA engine that ingests a train's live running state and continuously re-predicts arrival time at every upcoming station, improving on the static-schedule-plus-buffer baseline, and exposing this through APIs consumable by a dashboard, a mobile-style UI, and a station-display simulation.

Success criteria for the 3-day deliverable

- A model that measurably beats the naive baseline (schedule + last known delay) on held-out simulated/historical journeys, reported as MAE in minutes per station.
- ETA updates dynamically as a train's simulated position/events change, without a full restart.
- A REST API (`/eta`, `/train/{id}`, `/station/{code}/arrivals`) that a dashboard and a mobile-style screen both consume live.
- A short, clear story for judges: problem → data → model → live demo → scalability argument.

2. Scope for the 3-Day Build

This is the most important section for a time-boxed build: it protects the team from scope creep. Anything not listed under In-Scope is explicitly deferred.

In Scope

- Synthetic-but-realistic train running simulator: N trains, multi-station routes, injected delay events (signal halts, speed restrictions, congestion) — used as the default data source (see Section 4).
- Optional plug-in for a real public dataset (e.g. data.gov.in train delay data, or a Kaggle IRCTC/train-delay dataset) if the team finds one usable in the first few hours — treated as a bonus data source, not a dependency.
- Feature engineering pipeline turning raw running-state into model-ready features per train per checkpoint.
- Two prediction paths: (a) baseline heuristic (schedule + delay + fixed recovery), (b) ML model (gradient-boosted regression per segment).
- FastAPI/Flask backend exposing prediction + simulation-control endpoints.
- React dashboard: live map/list of trains, predicted vs scheduled ETA per station, delay trend chart.
- A short passenger-facing mobile-style screen (can be the same React app, responsive view) showing ETA for a selected train/PNR-style lookup.
- A basic station-display simulation view (arrivals board style) driven by the same API.
- Model evaluation notebook/report (MAE/RMSE vs baseline, by station and by horizon).

Out of Scope (explicitly deferred)

- Integration with actual NTES / Control Office Application (COA) systems — no real-time IR data access in a hackathon setting; architecture instead shows a clearly marked plug-in point.
- True GPS hardware / IoT ingestion — simulated position feed instead.
- Crew scheduling, platform allocation, and cleaning-ops optimization — mentioned as downstream consumers of the API, not built.
- Authentication, multi-tenant zone management, production-grade horizontal scaling, and formal load testing.
- Deep learning / sequence models (e.g. LSTM/Transformer) — explicitly deferred to "stretch goal" given 3-day budget; gradient boosting is the primary model.

3. System Architecture

The system is a linear pipeline: data sources feed an ingestion/feature layer, which feeds two prediction paths behind one API, consumed by three demo front-ends. The grey box marks where real Indian Railways feeds would plug in later without changing anything downstream — this is the key point to make to judges: the simulator is a stand-in for a data source, not a structural limitation.

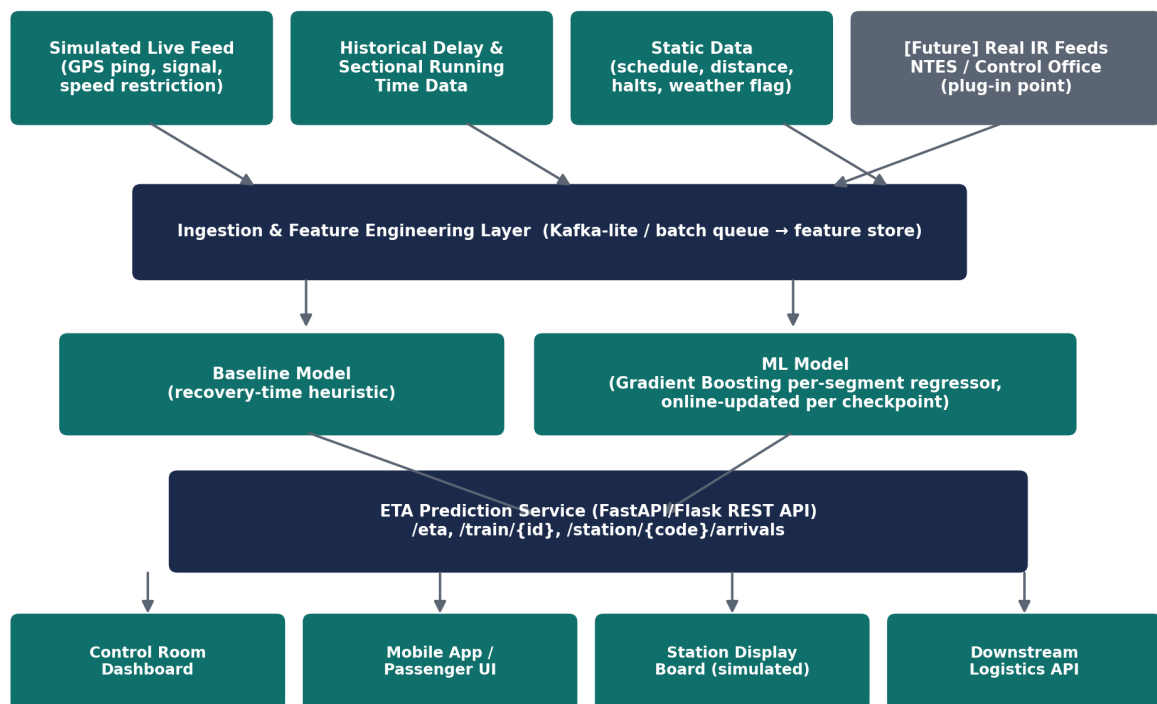


Figure 1. End-to-end architecture, from data sources to demo consumers.

Component notes

- **Simulator:** a background process advances each train along its route on a tick (e.g. every 2–5 sec = simulated minutes), emitting position, current delay, and occasional injected events.
- **Feature store:** can be as simple as an in-memory dict / SQLite table keyed by (train_id, checkpoint) for a 3-day build — no need for real Kafka/Redis unless the team already knows them well.
- **Baseline model:** ETA = scheduled arrival + current delay – recovery time already built into the schedule. Always computed, always shown, so the demo can visibly show "ML beats baseline."
- **ML model:** one gradient boosting regressor (e.g. LightGBM/XGBoost) predicting *remaining minutes to next station*, applied segment-by-segment and summed with uncertainty widening over horizon; retrained or updated in batch every so often on freshly simulated history.

- **API layer:** thin FastAPI/Flask service; same endpoints serve dashboard, mobile view, and station display so there is exactly one source of truth.

3.1 Tech Stack Summary

Layer	Technology
Language	Python 3 (backend, simulator, ML), JavaScript/TypeScript (frontend)
Backend / API	FastAPI (preferred) or Flask — REST endpoints per Section 6
Data handling	pandas (DataFrame pipelines, joins, rolling/groupby features), numpy (vectorized feature math, uncertainty bands, simulator noise)
ML modeling	scikit-learn (train/test split, metrics: MAE/RMSE), LightGBM or XGBoost (gradient boosting regressor)
Data storage	SQLite or in-memory store for the 3-day build — deliberately minimal, no managed DB/queue needed
Frontend	React (dashboard, passenger-style ETA view, station arrivals board), plain CSS or a lightweight component library
Dev / demo tooling	Jupyter/Colab notebook for model evaluation report, Git for version control, a simple background scheduler (e.g. Python threading/APScheduler) to drive the simulator tick
Stretch	Small LSTM (PyTorch or TensorFlow) only as a comparison slide, not the demo path

Chosen deliberately for a 3-day build: every piece here is something the team already listed as its comfort zone, with no new infra (no Kafka/Redis/managed DB) introduced purely for architectural elegance.

4. Data & Feature Design

Core entities: **Train** (id, type, route), **Route** (ordered list of stations with scheduled distance/time), **Journey** (one running instance of a train on a date), and **Event** (timestamped occurrence: halt, speed restriction, congestion flag, weather flag) attached to a Journey.

Feature list (per train, per checkpoint)

Group	Feature	Description
Static	distance_to_go_km	Remaining route distance to target station
Static	scheduled_time_to_go_min	Time-table implied remaining minutes
Static	num_intermediate_halts	Scheduled stops between here and target
Dynamic	current_delay_min	Delay at last reporting checkpoint
Dynamic	delay_trend_3pt	Slope of delay over last 3 checkpoints (worsening/recovering)
Dynamic	active_speed_restriction_flag	Whether a TSR is active on the next segment
Dynamic	congestion_score_downstream	Proxy for traffic density on the section ahead
Historical	hist_avg_delay_this_section	Mean historical delay for this train/section/weekday
Historical	hist_recovery_rate_section	Typical minutes recovered per 100km on this section
Context	weather_flag	Fog/rain flag (simulated, seasonal)
Context	day_type	Weekday / weekend / festival-season flag

Target variable: minutes-to-arrival at the next station (short horizon) and at the final/queried station (long horizon, computed by chaining segment predictions). Training data: simulator runs logged across many synthetic "days" with randomized event injection, so the model sees enough delay/recovery patterns to generalize.

5. Modeling Approach

Baseline (must-have, Day 1)

ETA = scheduled_arrival + current_delay – scheduled_recovery_time. This mirrors what the problem statement says Railways does today; it is the number the ML model must beat, and it must always be computable even if the ML service is down.

Primary model (core deliverable)

- Gradient boosting regression (LightGBM or XGBoost) trained per-segment to predict minutes-to-next-checkpoint, given the feature set above.
- For a multi-station ETA, chain segment predictions forward and sum, widening a simple uncertainty band with each additional segment (e.g. $\pm(k \times \text{segments})$ minutes) so the UI can show a confidence range, not a false-precision point estimate.
- Re-fit on a schedule (e.g. once per simulated day, or on-demand) using freshly accumulated simulated journeys — this is what lets the team credibly say "continuously refines its predictions."
- Evaluate with MAE and RMSE in minutes, broken down by: (a) horizon (1 station ahead vs 5 stations ahead), (b) with vs without an active disruption event — this breakdown is what makes the results slide credible to judges.

Stretch (only if time remains after core works end-to-end)

- Simple online/incremental update: nudge predictions using an exponentially-weighted recent-error correction per train, so ETA visibly tightens as a train gets closer to a station, without a full retrain.
 - A lightweight sequence model (e.g. small LSTM) as a comparison point — only as a "we explored this too" slide, not the demo path.
-

6. API Design

Method	Endpoint	Purpose
GET	/trains	List all simulated trains currently running, with summary status.
GET	/train/{train_id}	Current position, delay, and full station-wise ETA list (baseline + ML) for one train.
GET	/train/{train_id}/eta/{station_code}	Predicted ETA + confidence band at one specific upcoming station.
GET	/station/{station_code}/arrivals	Arrivals board: all trains due at a station soon, sorted by predicted ETA.
POST	/simulate/event	Inject a demo event (delay, TSR, congestion) into a running train, for live demo control.
GET	/model/metrics	Latest evaluation metrics (MAE/RMSE by horizon) for the control-room / judge-facing view.

All three front-ends (dashboard, passenger view, station display) read from the same endpoints; the /simulate/event endpoint is what makes the live demo compelling — you inject a delay on stage and watch every consumer update within seconds.

7. Day-by-Day Execution Plan

Written for a team of 3–5. Adjust parallelization to your actual headcount; the ordering and dependency logic (simulator before features, features before model, baseline before ML, API before front-end) should hold regardless of team size.

Day 1 — Foundations: simulator, baseline ETA, skeleton API/UI

Time	Owner	Task
Hr 0–1	All	Finalize scope (this doc), assign owners: Simulator, Model, API, Frontend.
Hr 1–4	Data/Sim owner	Build train/route/journey data model + simulator that advances trains and injects events.
Hr 1–4	API owner	Scaffold FastAPI/Flask app, wire to simulator state, stub baseline ETA endpoint.
Hr 4–7	Model owner	Generate first batch of simulated history; build feature extraction; implement baseline heuristic.
Hr 7–9	Frontend owner	Scaffold React app; static train list + station arrivals view hitting stub API.
Hr 9–10	All	Checkpoint: baseline ETA visible end-to-end (simulator → API → UI) even if ugly.

Day 2 — ML model, full API surface, live dashboard

Time	Owner	Task
Hr 0–3	Model owner	Train first gradient boosting model on simulated history; compute MAE vs baseline.
Hr 3–5	Model owner	Add segment-chaining for multi-station ETA + uncertainty band; wire /model/metrics.
Hr 0–5	API owner	Implement full endpoint set incl. /simulate/event; connect ML predictions.
Hr 0–6	Frontend owner	Build live-updating dashboard (poll or WebSocket), delay trend chart, event-injection button.
Hr 6–9	Frontend owner	Passenger-style ETA lookup screen + station arrivals board view.
Hr 9–10	All	Checkpoint: ML ETA beats baseline visibly on at least one prepared demo scenario.

Day 3 — Evaluation, polish, rehearsal, submission

Time	Owner	Task
Hr 0–3	Model owner	Model evaluation report: MAE/RMSE by horizon and by disruption vs no-disruption.
Hr 0–3	Frontend owner	Visual polish: consistent styling, loading states, mobile-responsive check.
Hr 3–5	All	Rehearse the live demo script end-to-end (Section 8); fix rough edges found in rehearsal.
Hr 5–6	All	Prepare slides: problem → architecture diagram → results table → scalability argument.

Time	Owner	Task
Hr 6–8	All	Buffer for bug fixes; record a backup demo video in case of live-demo failure.
Hr 8–9	All	Final rehearsal + submission packaging.

8. Demo Script (for judges)

- **Open (30s):** state the problem in one sentence — static schedule + buffer vs. real-time conditions — using the problem statement's own framing.
- **Show architecture (30s):** Figure 1, pointing at the marked real-data plug-in point to pre-empt "is this real data?" questions.
- **Live demo (2 min):** open dashboard showing several trains running; pick one train, show its baseline ETA vs ML ETA side by side; hit /simulate/event to inject a delay live; watch the ETA update within seconds across dashboard, passenger view, and station board simultaneously.
- **Results (30s):** show the MAE table — ML model's error in minutes vs baseline's error, especially during the injected-disruption scenario, since that is exactly where static schedules fail.
- **Close (30s):** scalability & extensibility argument (Section 9) and one-line summary of what real IR integration would require.

9. Scalability Argument (for the pitch, not the build)

You will not build 24x7-scale infra in 3 days — you need a credible *argument*, not a load test. Make these points verbally/in slides:

- The per-train feature computation is independent across trains → horizontally scalable (one worker per shard of trains, e.g. by zone).
- The model is a single small gradient boosting model applied per segment, not per train — cheap to serve and easy to scale to thousands of concurrent trains.
- Batch retraining decouples model-update cost from prediction-serving cost, so serving stays fast even as the model improves over time.
- The zone-wise operational structure of Indian Railways maps naturally onto zone-sharded deployment of this same pipeline.

10. Risks & Mitigations

Risk	Likelihood	Mitigation
No usable real dataset found in time	Medium	Default plan is synthetic simulation from hour 1; real data is a bonus, never a blocker.
ML model underperforms baseline	Medium	Keep baseline always visible/computed; report honestly on horizons where ML wins vs. not, rather than hiding it.
Live demo fails on stage	Medium	Record a backup demo video by end of Day 3; rehearse the event-injection flow at least 3 times.
Team stack mismatch mid-build	Low	Stack fixed upfront (Python/Flask-FastAPI + React); no new frameworks introduced after Day 1.
Scope creep (real-time infra, auth, etc.)	Medium	Section 2's out-of-scope list is the standing answer to "should we also build X".

11. Stretch Goals (only after core demo works)

- Per-train confidence visualization (fan chart of possible arrival times, widening with horizon).
- Simple crew/platform impact note: "if this train is +12 min late, platform X is double-booked" as a downstream-planning teaser.
- Small LSTM/sequence-model comparison slide.
- One real public dataset wired in alongside the simulator, clearly labeled as such.

End of document. This PRD is scoped deliberately small for a 3-day build; the architecture and API are designed so a real Indian Railways data integration could be swapped in later without changing the model, API, or front-end contracts.